VNUHCM - UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Sorting Algorithms Project Report

Group - 10

Participants:

Full name	ID	Tasks	Contribution (%)
Huỳnh Phúc Nguyên	25127106	Implementation and Command line arguments	
Đỗ Đăng Tú	25127167		
Đỗ Minh Khang	25127363	Report and Proofreading report	
Huỳnh Đình Khương	25127396	Experiment and Visualization	
Total			100

Table of contents

Table of contents 2
1. Introduction 3
2. Algorithms 3
2.1. Bubble Sort 3
2.2. Shaker Sort 4
2.3. Selection Sort 5
2.4. Insertion Sort 7
2.5. Binary Insertion Sort 8
2.6. Shell Sort 9
2.7. Heap Sort 10
2.8. Merge Sort 12
2.9. Quick Sort 13
2.10. Counting Sort 15
2.11. Radix Sort 16
2.12. Flash Sort 17
3. Experiment 19
4. Project Organization 27
5. References 27

1. Introduction

This report aims to provide insights into sorting including several common as well as widely-used sorting algorithms. The report examinees 12 distinct algorithms, namely: Selection Sort, Insertion Sort, Shell Sort, Bubble Sort, Heap Sort, Merge Sort, Quick Sort, Radix Sort, Counting Sort, Binary Insertion Sort, Shaker Sort and Flash Sort. For each algorithm, the core concept, explanation, complexity analysis, and other relevant information will be presented along with the real-time experimental results. All the algorithms are implemented to sort integer values in ascending order.

The project also includes simple C++ files to run the experiments. The ideas behind these files and details about the organization of the source code will also be described in this report.

2. Algorithms

2.1. Bubble Sort

2.1.1. Core concept

Bubble Sort is a sorting algorithm that repeatedly compares two adjacent elements and swaps them if they are in the wrong order. This process continues until the entire list is sorted.

2.1.2. Explanation

- Traverse the list one element at a time
- Compare each element with its next. If the current element is larger, swap their positions. After each pass, the largest element will move to the end of the list.
- Repeat this process until the entire list is sorted.

2.1.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass:
    - List (before): [***23***, 1, 10, 5, 2]
    - List (after): [1, 10, 5, 2, ***23***]
    - Explain: '23' is the largest element. Therefore, its "bubbles" to the end.

- 2nd pass:
    - List (before): [1, ***10***, 5, 2, **23**]
    - List (after): [1, 5, 2, ***10***, **23**]
    - Explain: '10' is the next largest element. Therefore, its "bubbles" to the end.
- 3rd pass:
    - List (before): [1, ***5***, 2, **10**, **23**]
    - List (after): [1, 2, ***5***, **10**, **23**]
    - Explain: '5' is the next largest element. Therefore, its "bubbles" to the end.

2.1.4. Complexity analysis

a) Time Complexity **All cases: O(n^2)** – Nested loops
b) Space complexity: **O(1)** – Sorting in-place and only need constant extra space

2.1.5. Other information

- Easy to understand and implement; works well with small lists.
- Can be optimized by adding a **swap flag**, which could make the **Best Case: O(n)**
- Has very practical real-world applications due to its poor efficiency compared to other algorithms.
- Forms the basis for some related algorithms such as **Shaker Sort (Cocktail Sort)**.
- It is a stable algorithm because it will preserve the relative order of the equal elements

2.2. Shaker Sort

2.2.1. Core concept

Shaker Sort is a variation of the Bubble Sort algorithm. Instead of traversing the list in only one direction, it moves in both directions alternately, which can improve the efficency of the original Bubble sort algorithm.

2.2.2. Explanation

- Traverse the list one element at a time
- Compare each element with the next element. If the current element is larger, swap their positions. After one forward pass, the largest element will move to the end of the list, similar to Bubble Sort.
- Traveres the list in the opposite direction and repeat the same process to move the smallest element to the beginning of the list.
- Repeat these forward and backward passes until the entire list is sorted.

2.2.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass:
  - List (before): [***23***, 1, 10, 5, 2]
  - List (after stage 1): [1, 10, 5, ***2***, ***23***]
  - List (after stage 2): [**1**, ***2***, 10, 5, **23**]
  - Explain: '23' is the largest element. Therefore, its "bubbles" to the end during the forward pass. Then the algorithm traverses backward and places **2** in its correct position near the beginning.
- 2nd pass:
  - List (before): [**1**, **2**, ***10***, 5, **23**]
  - List (after stage 1): [**1**, **2**, ***5***, ***10***, **23**]
  - List (after stage 2): [**1**, **2**, ***5***, **10**, **23**]
  - Explain: **10** moves to its correct position during the forward pass, and **5** is placed correctly during the backward pass.

2.2.4. Complexity analysis

a) Time Complexity **All case: $O(n^2)$** – Similar to Bubble sort
b) Space complexity: **O(1)** – Sorting in-place and only need constant extra space

2.2.5. Other information

- More efficient than Bubble sort in certain cases, especially when the list conatins a small number of unsorted elements near either end.
- It is a stable algorithm because it preserves the relative order of the equal elements
- The algorithm is known by several other names, including **Cocktail Sort**, **Shuttle Sort**, and **Ripple Sort**.

2.3. Selection Sort

2.3.1. Core concept

Selection Sort is a simple comparison-based sorting algorithm that works by dividing the list into two parts: sorted and unsorted. The algorithm repeatedly selects the smallest element from the unsorted portion and swaps it with the first element of the unsorted part, thereby extending the sorted portion of the list.

2.3.2. Explanation

- Find the smallest element in the unsorted portion of the list
- Swap that element with the first element of the unsorted part (which becomes the next element in the sorted part).

- Repeat this process until the entire list is sorted.

2.3.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass:
  - List (before): [23, ***1***, 10, 5, 2]
  - List (after): [***1***, 23, 10, 5, 2]
  - Explain: '1' is the smallest unsorted element. Therefore, it is placed at the beginning of the unsorted part.
- 2nd pass:
  - List (before): [**1**, 23, 10, 5, ***2***]
  - List (after): [**1**, ***2***, 10, 5, 23]
  - Explain: '2' is the smallest unsorted element. Therefore, it is placed at the beginning of the unsorted part
- 3rd pass:
  - List (before): [**1**, **2**, 10, ***5***, 23]
  - List (after): [**1**, **2**, ***5***, 10, 23]
  - Explain: '5' is the smallest unsorted element. Therefore, it is placed at the beginning of the unsorted part
- 4th pass:
  - List (before): [**1**, **2**, **5**, ***10***, 23]
  - List (after): [**1**, **2**, **5**, ***10***, 23]
  - Explain: '10' is already the smallest unsorted element. Therefore, it remains at beginning of the unsorted part

2.3.4. Complexity analysis

a) Time Complexity **All case: $O(n^2)$** – The algorithm always scans the remaining unsorted elements.

b) Space complexity: **O(1)** – In-place sorting algorithm which requires only constant extra space.

2.3.5. Other information

- Easy to understand and implement; suitable for small lists.
- Often used for educational purposes because it demonstrates the fundamental concept of selection-based sorting.
- The selection idea is related to other algorithms such as **Heap Sort**, which improves efficiency by using a heap data structure.
- Selection Sort is **not a stable sorting algorithm**, because swapping elements may change the relative order of equal elements.

2.4. Insertion Sort

2.4.1. Core concept

Insertion Sort is a simple sorting algorithm that works by dividing the list into two parts: sorted and unsorted. The algorithm iteratively inserts each element from the unsorted part into its correct position in the sorted part.

2.4.2. Explanation

- Assume the first element of the list is already sorted
- Take the first element of the unsorted part, search for its correct position in the sorted part and insert it there.
- Repeat the process until the entire list is sorted.

2.4.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass:
  - List (before): [**23**, ***1***, 10, 5, 2]
  - List (after): [***1,*** **23**, 10, 5, 2]
  - Explain: '1' is smaller than '23'. Therefore, it is inserted before it.
- 2nd pass:
  - List (before): [**1**, **23**, ***10***, 5, 2]
  - List (after): [**1,** ***10***, **23**, 5, 2]
  - Explain: '10' is smaller than '23' but larger than '1'. Therefore, it is inserted between them.
- 3rd pass:
  - List (before): [**1**, **10**, **23**, ***5***, 2]
  - List (after): [**1**, ***5***, **10**, **23**, 2]
  - Explain: '5' is smaller than '10' but larger than '1'. Therefore, it is inserted between them.
- 4th pass:
  - List (before): [**1**, **5**, **10**, **23**, ***2***]
  - List (after): [**1**, ***2***, **5**, **10**, **23**]
  - Explain: '2' is smaller than '5' but larger than '1'. Therefore, it is inserted between them.

2.4.4. Complexity analysis

a) Time Complexity:

- **Worst Case: $O(n^2)$** – Occurs when the input array is sorted in reverse order.
- **Best Case: O(n)** – Occures when the array is already sorted.
- **Average Case: $O(n^2)$** – Contains 2 nested loops for comparison and swapping.

b) Space complexity: **O(1)** – Insertion Sort is an **in-place sorting algorithm** and requires only constant extra space.

2.4.5. Other information

- Easy to understand and implement; works well with small or nearly sorted list
- It is a **stable sorting algorithm**, meaning it preserves the relative order of equal elements.
- Can be optimized by shifting the values instead of searching for valid position
- It serves as the basis for several other algorithms such as **Shell Sort** and **Binary Insertion Sort**.
- It is commonly used in **hybrid sorting algorithms** for handling small subarrays.

2.5. Binary Insertion Sort

2.5.1. Core concept

Binary Insertion Sort is a variation of the traditional Insertion Sort. Instead of using a linear search to find the correct insertion position, it uses **Binary Search**, since the sorted portion of the list is already ordered.

2.5.2. Explanation

- Assume the first element of the list is already sorted
- Take the first element from the unsorted part and use **Binary Search** to find its correct position in the sorted part.
- Insert the element into that position.
- Repeat this process until the entire list is sorted.

2.5.3. Manual Run

The process is similar to Insertion Sort; however, the position of the element in the sorted part is determined using **Binary Search** instead of a linear scan.

2.5.4. Complexity analysis

a) Time Complexity:
- **Worst Case: $O(n^2)$** – Occurs when the array is sorted in reverse order.
- **Best Case: O(n)** – Occurs when the array is already sorted.

- **Average Case: $O(n^2)$** – Although searching is faster, elements still need to be shifted for insertion.

b) Space complexity: **O(1)** – The algorithm sorts the list **in-place** and requires only constant additional space.

2.5.5. Other information

- The overall time complexity remains **$O(n^2)$** because shifting elements still requires linear time.
- However, the **searching complexity** for finding the insertion position is reduced from **O(n)** to **O(log n)** using Binary Search.
- This optimization can reduce the number of comparisons, although the total running time improvement is usually small.

2.6. Shell Sort

2.6.1. Core concept

Shell Sort is a sorting algorithm that improves upon the traditional **Insertion Sort**. Instead of comparing only adjacent elements, it compares elements that are a certain distance apart using a **gap sequence**. The gap decreases after each iteration. By initially comparing elements that are far apart, the algorithm can significantly improve performance compared to standard Insertion Sort.

2.6.2. Explanation

- Choose a gap sequence (usually n/2, n/4, ..., 1).
- For each gap, perform **Insertion Sort** on elements that are that distance apart.
- Reduce the gap and repeat the process until the gap becomes **1**, which results in a final Insertion Sort pass on the entire list.

2.6.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass (gap = n/2 = 2):
  - List (before): [***23***, ***1***, ***10***, ***5***, ***2***]
  - List (after): [***2***, ***1***, ***10***, ***5***, ***23***]
  - Explanation: Elements that are two positions apart are sorted using Insertion Sort.
- 2nd pass (gap = n/4 = 1):

- List (before): [*2*, *1*, *10*, *5*, *23*]
- List (after): [*1*, *2*, *5*, *10*, *23*]
- Explanation: When the gap becomes **1**, the algorithm performs a standard Insertion Sort on the entire list.

2.6.4. Complexity analysis

a) Time Complexity:
   - **Worst Case: less than $O(n^2)$** – Depends heavily on the gap sequence used.
   - **Best Case: O(n.log n)** – The input array is already sorted.
   - **Average Case: $O(n^{\sim 1.25})$** – Depends on the chosen gap sequence

b) Space complexity: **O(1)** – Shell Sort is an **in-place sorting algorithm** and requires only constant extra space.

2.6.5. Other information

- Shell Sort is considered one of the earliest algorithms to improve upon the **$O(n^2)$** performance of simple sorting algorithms such as Insertion Sort.
- It is **not a stable sorting algorithm**, since elements may move across long distances during sorting.
- The exact time complexity of Shell Sort is difficult to determine because it depends on the **gap sequence** used.
- According to **Poonen's Theorem**, the worst-case complexity of Shell Sort lies between **$\Theta((n \log n)^2 / (\log \log n)^2)$** and **$\Theta(n(\log n)^2)$**.

2.7. Heap Sort

2.7.1. Core concept

Heap Sort is a sorting algorithm that uses a special binary tree structure called a **heap**. The given list is first rearranged to satisfy the heap property, and then the root node (which contains the largest element in a max-heap) is repeatedly removed and placed at the end of the list.

2.7.2. Explanation

- Convert the list into a max-heap
- Swap the root element with the last element of the heap.
- Remove the last element (which is the largest value) from the heap and reduce the heap size by one. Then, restore the heap property using **heapify**.
- Repeat this process until the heap size becomes 1.

2.7.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- 1st pass:
  - List (before): [23, ***1***, 10, ***5***, 2]
  - List (heapify): [23, ***5***, 10, ***1***, 2]
  - List (after): [2, 5, 10, 1, **23**]
  - Explanation: The list is converted into a heap. The largest value (**23**) is swapped with the last element, and the heap size is reduced by one.
- 2nd pass:
  - List (before): [***2***, 5, ***10***, 1, **23**]
  - List (heapify): [***10***, 5, ***2***, 1, **23**]
  - List (after): [1, 5, 2, **10**, **23**]
  - Explanation: The remaining list is heapified again. The largest value (**10**) is swapped with the last element of the heap.
- 3rd pass:
  - List (before): [***1***, ***5***, 2, **10**, **23**]
  - List (heapify): [***5***, ***1***, 2, **10**, **23**]
  - List (after): [2, 1, **5**, **10**, **23**]
  - Explanation: The remaining list is heapified again. The largest value (**5**) is swapped with the last element of the heap.

- 4th pass:
  - List (already a heap): [***2***, ***1***, **5**, **10**, **23**]
  - List (after): [**1**, **2**, **5**, **10**, **23**]
  - Explanation: The remaining list already satisfies the heap property. The largest value (**2**) is swapped with the last element.

2.7.4. Complexity analysis

a) Time Complexity **All case: O(n.log n)** – Due to the heap nature
b) Space complexity: **O(log n)** – Recursive the heap's function – heapify

2.7.5. Other information

- Heap Sort can be considered an optimization of the **Selection Sort concept**, where the largest element is repeatedly selected efficiently using a heap structure.
- It is **not a stable sorting algorithm**, since swapping elements may change the relative order of equal elements.
- The space complexity can be reduced to **O(1)** if an iterative heapify implementation is used.

- Although the time complexity is **O(n.log n)**, the constant factors are relatively larger compared to some other algorithms with the same complexity, such as Quick Sort.

2.8. Merge Sort

2.8.1. Core concept

Merge Sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach by recursively dividing the input array into two halves, sorting each half, and then merging the sorted halves back together.

2.8.2. Explanation

- Recursively split the list into two halves until each sub-list contains only one element.
- Merge each the sub-lists together in sorted order.
- Continues until all sub-lists are fully merged

2.8.3. Manual Run

Given list: [23, 1, 10, 5, 2], n = 5

- Split: [23, 1, 10, 5, 2] → [23, 1] [10, 5, 2] → [23] [1] [10] [5, 2] → [23] [1] [10] [5] [2]
- 1st pass: [23] [1] [10] [2, 5]
- 2nd pass: [1, 23] [2, 5, 10]
- 3rd pass: [1, 2, 5, 10, 23]

2.8.4. Complexity analysis

a) Time Complexity **All case: O(n.log n)** – Divide and Merge.
b) Space complexity: **O(n)** – additional space is required for temporary arrays during the merge process.

2.8.5. Other information

- Merge Sort is one of the easiest **O(n log n)** algorithms to understand and is commonly used for teaching divide-and-conquer techniques.
- It performs very well with **Linked List** data structures.
- It is a **stable sorting algorithm** which preserves the relative order of equal elements.
- The algorithm can be generalized to divide the array into **k sub-lists** instead of two. In that case, the time complexity becomes **O(n $\log_k$ n)**, where **k** is the number of sub-lists.

2.9. Quick Sort

2.9.1. Core concept

Quick Sort is a sorting algorithm based on the Divide and Conquer technique. It selects an element as a **pivot** and partitions the given array around that pivot by moving smaller elements to its left and larger elements to its right. The algorithm then recursively applies the same process to the two resulting sublists.

2.9.2. Explanation

- Choose a pivot element. There are many ways to select a pivot, but in this report the **last element** is used.
- Partition the list. Several partitioning methods exist; in this report, **Hoare's partition scheme** is used.
    - Use two pointers: one starting from the beginning of the list and another from the second-to-last element (excluding the pivot).
    - Move the pointers toward each other until a pair of elements that should be swapped is found.
    - Swap those elements and continue the process.
    - If the pointers cross, swap the pivot element with the element at the first pointer's position.
- Recursively apply the same process to the two sublists until each sublist contains only one element.

2.9.3. Manual Run

Given list: [23, 1, 10, 5, 2, 18, 7], n = 7

- 1st pass:
  - List (before): [23, 1, 10, 5, 2, 18, ***7***]
  - List (after): [2, 1, 5, ***7***, 23, 18, 10]
  - Explanation: Choose '7' as pivot, partition the list and place '7' in the correct position.
- 2nd pass:
  - List (before): [2, 1, ***5***, **7**, 23, 18, ***10***]
  - List (after): [2, 1, ***5***, **7**, 18, ***10***, 23]
  - Explain: Choose '5' and '10' as pivots for the sub-lists and partition them.
- 3rd pass:
  - List (before): [2, ***1***, **5**, **7**, ***18***, ***10***, ***23***]
  - List (after): [***1***, 2, **5**, **7**, ***18***, ***10***, ***23***]
  - Explain: Choose **1**, **18**, and **23** as pivots for the remaining sublists and place them in their correct positions.

2.9.4. Complexity analysis

a) Time Complexity:
   - **Worst Case: O(n^2)** – Occurs when the worst pivot is chosen (e.g., when the list is already sorted and the last element is used as the pivot).
   - **Best Case: O(n.log n)** – Occurs when the pivot divides the list into two equal halves.
   - **Average Case: O(n.log n)** – Depends on the pivot selection method.

b) Space complexity:
   - **Worst Case: O(n)** – Deep recursion leads to a large call stack.
   - **Best Case: O(log n)** – Balanced recursion depth.
   - **Average Case: O(log n)** – Depends on pivot selection.

2.9.5. Other information

- Often considered one of the **fastest general-purpose sorting algorithms** in practice.
- Performs well with **large datasets**, but may not be ideal for very small arrays.
- Frequently used in **hybrid sorting algorithms**.
- By using some smarter techniques for choosing the pivot
- Quick Sort is **not a stable sorting algorithm** because swapping elements may change the relative order of equal elements.
- **Tail call optimization** can reduce the worst-case space complexity to **O(log n)**.

2.10. Counting Sort

2.10.1. Core concept

Counting Sort is a sorting algorithm that works by counting the number of occurrences (frequencies) of each unique element in the input list. These counts are stored in an auxiliary array, and the sorted output is generated using the prefix sums of the count array.

2.10.2. Explanation

- Find the maximum element in the list and create a count array cntArr[] of size max + 1, initialized to 0.
- Traverse the input list and increment cntArr[i] each time the element i appears.
- Compute the **prefix sum** of the count array.
- Create a result array res [] of the same size as the input.
- Traverse the input list from the end and place each element in its correct position using the count array.

2.10.3. Manual Run

Given list: [3, 2, 2, 3, 1], n = 5

- Max element = 3, create count array: [0, 1, 2, 2]
- Calculate prefix sums: [0, 1, 3, 5]
- Create result list of size n: [0, 0, 0, 0, 0]
- Traverse the input list from end and update:
    - Last element: '1', at index '1' from the count array, the value is '1'. Take the value '1' and put it at index '1-1' in the result array, decrease the value by 1 from count array
        - Result list: [**1**, 0, 0, 0, 0]
        - Count array: [0, 0, 3, 5]
    - Second last element: '3', at index '3' from the count array, the value is '5'. Take the value '3' and put it at index '5-1' in the result array, decrease the value by 1 from count array
        - Result list: [**1**, 0, 0, 0, **3**]
        - Count array: [0, 0, 3, 4]
    - Middle last element: '2', at index '2' from the count array, the value is '3'. Take the value '2' and put it at index '3-1' in the result array, decrease the value by 1 from count array
        - Result list: [**1**, 0, **2**, 0, **3**]
        - Count array: [0, 0, 2, 4]

- Second element: '2', at index '2' from the count array, the value is '2'. Take the value '2' and put it at index '2-1' in the result array, decrease the value by 1 from count array
    - Result list: [**1**, **2**, **2**, 0, **3**]
    - Count array: [0, 0, 1, 4]
- First element: '3', at index '3' from the count array, the value is '4'. Take the value '3' and put it at index '4-1' in the result array, decrease the value by 1 from count array
    - Result list: [**1**, **2**, **2**, **0**, **3**]
    - Count array: [0, 0, 1, 3]

2.10.4. Complexity analysis

a) Time Complexity **All cases: O(n + k)** – Iterating through the input, count array and result
b) Space complexity: **O(n + k)** – Space for the result list and count array

2.10.5. Other information

- Counting Sort is a **non-comparison-based sorting algorithm**.
- Works efficiently when the **range of input values is not significantly larger than the number of elements**.
- Usually applied to **non-negative integer datasets**.
- Commonly used as a **subroutine in Radix Sort**.

2.11. Radix Sort

2.11.1. Core concept

Radix Sort is a linear sorting algorithm that sorts elements by processing them digit by digit or character by character. It repeatedly sorts the elements from the **least significant digit/character** to the **most significant digit/character** using a stable sorting algorithm, typically **Counting Sort**.

2.11.2. Explanation

- If the elements have different lengths, find the element with the most digits and pad the shorter elements if necessary.
- Sort the list by the **least significant digit (LSD)** using Counting Sort while preserving the relative order of elements.
- Move to the next digit and repeat the process until the **most significant digit (MSD)** has been processed

2.11.3. Manual Run

Given list: [233, 841, 104, 55, 322, 718, 7], n = 7

- Max digit = 3, List (padding): [233, 841, 104, 055, 322, 718, 007]
- 1st pass – Working with LSD:
  - List (before): [23***3***, 84***1***, 10***4***, 05***5***, 32***2***, 71***8***, 00***7***]
  - List (after): [84***1***, 32***2***, 23***3***, 10***4***, 05***5***, 00***7***, 71***8***]
  - Explain: Sort by the rightmost digit while maintain their relatvive order
- 2nd pass – Working with the next LSD:
  - List (before): [8***4***1, 3***2***2, 2***3***3, 1***0***4, 0***5***5, 0***0***7, 7***1***8]
  - List (after): [1***0***4, 0***0***7, 7***1***8, 3***2***2, 2***3***3, 8***4***1, 0***5***5]
  - Explain: Sort by the second rightmost digit while maintain theit relatvive order
- 3rd pass – Working with MSD:
  - List (before): [***1***04, ***0***07, ***7***18, ***3***22, ***2***33, ***8***41, ***0***55]
  - List (after): [***0***07, ***0***55, ***1***04, ***2***33, ***3***22, ***7***18, ***8***41,]
  - Explain: Sort by the leftmost digit while maintain their relatvive order

2.11.4. Complexity analysis

a) Time Complexity **All case: O(d.(n+k))** – Always need to cycle through all **d** digits and using Count Sort
b) Space complexity: **O(n + k)** – Countarray for Count Sort

2.11.5. Other information

- Radix sort is a non-compairison-based sorting algorithm.
- Works well with **large sets of non-negative integers.**
- It is **stable** because it relies on a stable sorting algorithm (Counting Sort).
- Other sorting algorithms may also be used as the internal sorting method, giving a complexity of **O(d·T(n))**, where **T(n)** is the complexity of the chosen algorithm.

2.12. Flash Sort

2.12.1. Core concept

Flash Sort is an **in-place sorting algorithm** designed to distribute data evenly across classes (buckets). It works similarly to **Bucket Sort** by first classifying elements into buckets and then sorting each bucket.

2.12.2. Explanation

- Find the maximum and minimum elements in the input list.
- Divide the the list into classes based on the value range and input size.

- Create a count array that stores the number of elements in each class.
- Calculate the prefix sum to determine class boundaries.
- Re-arrange the input element into their corresponding classes and sort each class by another sorting algorithm.

2.12.3. Manual Run

Given list: [3, 1, 9, 7, 5], n = 5

- Find: Min = 1, Max = 9
- Create classes:
  - Number of classes - m: Usually m = ~0,45n. For simplicity, let's use m = 3.
  - Calculate which class each element belongs by using following formula:

$$K(A[i]) = \left\lfloor \frac{(m-1) \times (A[i] - Min)}{Max - Min} \right\rfloor$$

- Compute the class for each element:
  - K(3) = 0
  - K(1) = 0
  - K(9) = 2
  - K(7) = 1
  - K(5) = 1
- Create count array: cnt = [2, 2, 1]
- Calculate upper bound index (rightmost idex for each class): cnt = [1, 3, 4]
- Re-arrange all the elements to there correct class using counting technique (similar to Counting Sort): [1, 3, 7, 5, 9] ⇔ Class 0: [1, 3], Class 1: [7, 5], Class 2: [9]
- Insertion Sort each class: Class 0: [1, 3], Class 1: [5, 7], Class 2: [9] ⇔ [1, 3, 5, 7, 9]

2.12.4. Complexity analysis

a) Time Complexity:
   - **Worst Case: O(n^2)** – Exist outlier
   - **Best Case: O(n)** – The classes have the same size
   - **Average Case: O(n)** – All element distributed evenly into classes

b) Space complexity: **O(m)** – Number of classes

2.12.5. Other information

- Flash Sort is **not a stable sorting algorithm**.
- It is a more **efficent way to implement Butket Sort**
- It **using many other sorting techniques such as Counting Sort and Insertion Sort**
- It is relatively **rarely used in practice**.

- Compared to other algorithms, **fewer academic resources and references are available**.

3. Experiment

3.1. Randomized data

Data order: Random												
Data size	10000		30000		50000		100000		300000		500000	
Resulting statics	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**
Heap sort	1.4748	235337	5.5577	800913	9.8357	1409652	19.7247	3019918	61.8898	10001122	108.928	17397675
Merge sort	3.1694	120309	10.3812	408507	20.1419	718135	37.0161	1536183	107.953	5084168	178.63	8837201
Quick sort	1.2654	155253	2.7089	524129	4.8252	918679	10.4284	2082652	32.0162	7155166	57.9746	13480869
Binary Insertion sort	18.2879	25423450	169.244	226898845	458.101	625601762	1817.1	2499298577	16795.2	22491393059	47501.7	62436482945
Counting sort	0.5794	70001	0.4215	210000	0.6794	332768	1.3195	632769	3.3517	1832769	5.0496	3032769
Bubble sort	88.1436	100009999	1327.16	900029999	3990	2500049999	20373.5	10000099999	201585	90000299999	573019	2.5E+11
Insertion sort	48.8648	50209153	400.312	451964906	1092.98	1248286583	4184.19	4966159515	39131.2	44999326549	419258	1.24947E+11
Selection sort	28.6185	100020001	253.943	900060001	735.673	2500100001	2808.96	10000200001	27858.2	90000600001	71288.4	2.50001E+11
Shell sort	3.0711	267827	9.2483	919861	15.9629	1835453	29.3856	4247205	94.9863	14453840	203.868	29793376
Shaker sort	146.953	100005001	1816.82	900015001	5745.39	2500025001	24743.4	10000050001	195522	90000150001	416450	2.5E+11
Radix sort	1.0924	9999	2.7429	29999	4.1451	49999	8.7554	99999	15.1937	299999	25.1184	499999
Flash sort	0.396	99038	0.7349	296410	2.0964	465123	2.1871	874526	8.3904	2602118	18.5112	4480435

Table 3.1. Randomized data

3.1.1. Running time

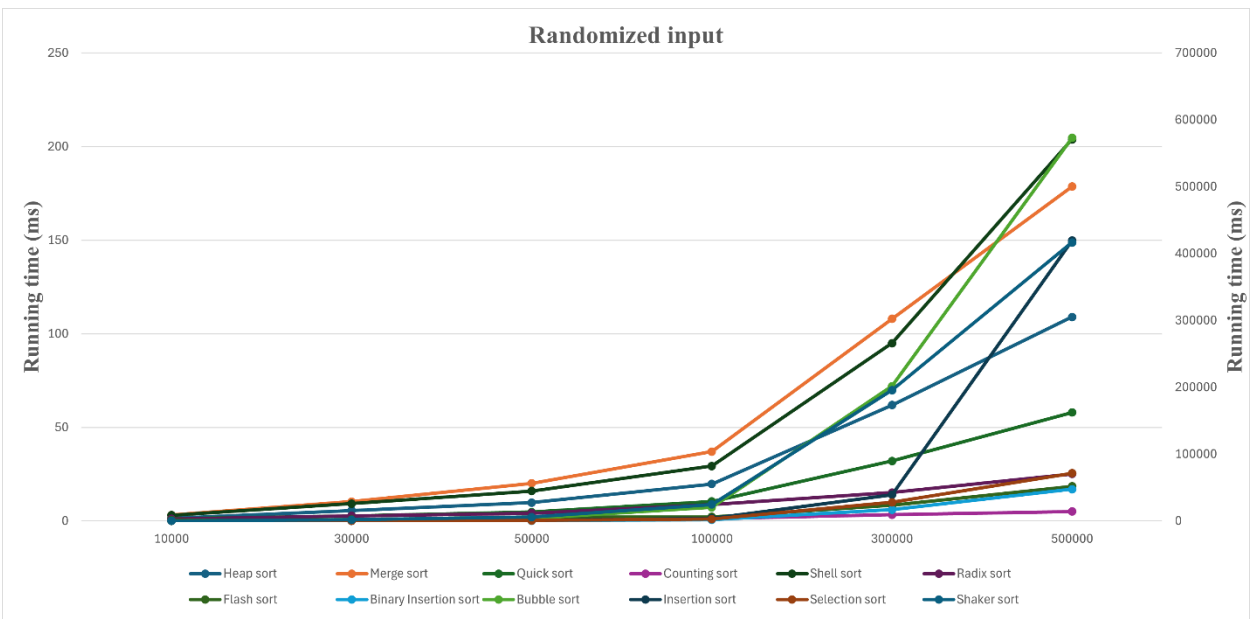

Chart 3.1.1. Randomized data-running time

- **Counting sort**, **Radix sort**, and **Flash sort** are the **fastest algorithms** as the data size increases.
- **Selection sort**, **Bubble sort**, and **Shaker sort** are the **slowest algorithms**.

- Quadratic algorithms (**Selection, Bubble, Shaker**) are slow because they repeatedly scan large portions of the array, leading to **O(n²)** growth.
- Algorithms like **Merge sort, Heap sort, and Quick sort** perform better with **O(n log n)** complexity, making them scalable for larger inputs.
- **Counting sort, Radix sort, and Flash sort** are the fastest since they operate in near **O(n)** time and do not rely heavily on comparisons.

3.1.2. Number of comparisons

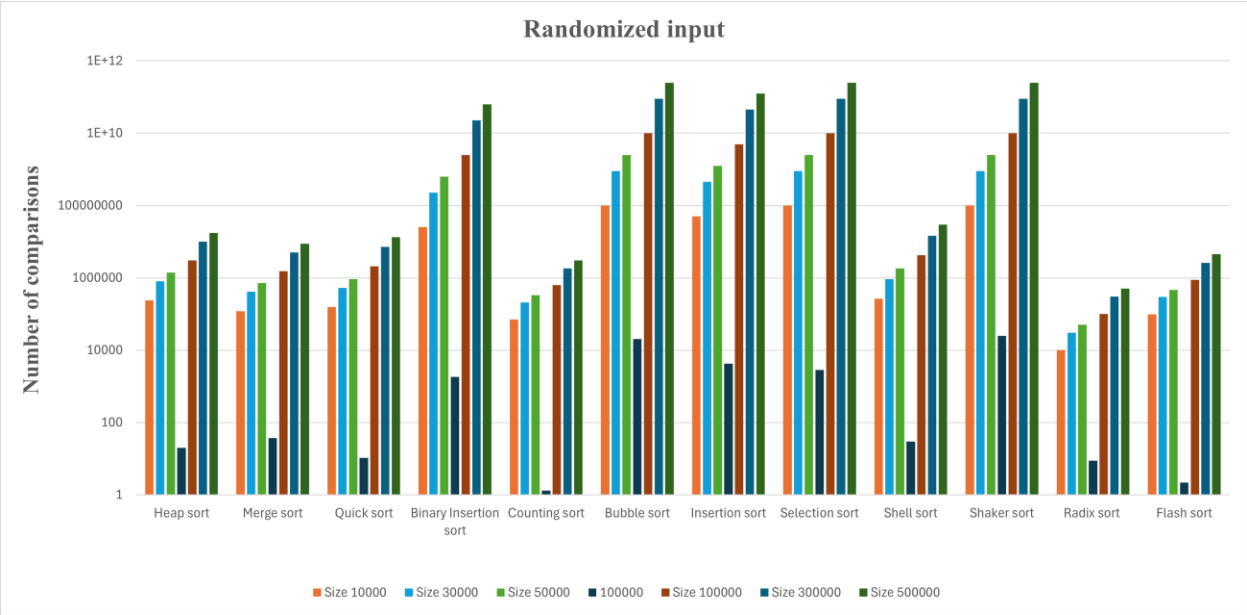

Chart 3.1.2. Randomized data-number of comparisons

- **Selection sort**, **Bubble sort**, and **Shaker sort** perform the **largest number of comparisons**.
- **Counting sort** and **Radix sort** perform the **fewest comparison**

- Quadratic algorithms perform a very large number of comparisons due to nested loops. **O(n log n)** algorithms reduce comparisons significantly by dividing the problem into smaller parts.
- **Counting sort and Radix sort** use almost no comparisons, as they are based on counting or digit processing rather than element comparison.

3.2. Nearly sorted data

Data order: Nearly sorted data												
Data size	10000	Size 10000	30000	Size 30000	50000	Size 50000	100000	Size 100000	300000	Size 300000	500000	Size 500000
Resulting statics	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**
Heap sort	1.2239	244448	3.6775	826340	7.2044	1455447	13.7138	3112518	44.4611	10279799	82.5298	17837784
Merge sort	2.5965	86921	9.5076	276922	17.522	448418	27.2316	907049	84.4964	2855912	143.97	4834182
Quick sort	0.6777	119399	1.1432	400854	2.5072	701495	4.7202	1494183	11.791	4894997	20.592	8498924
Binary Insertion Sort	0.7169	328910	1.25	1172504	1.2972	1673328	2.6922	3466178	8.6593	11058492	31.8469	19102184
Counting sort	0.3502	70001	0.356	210001	0.6124	350001	1.4544	700001	3.6743	2100001	6.9273	3500001
Bubble sort	37.8295	100009999	349.788	900029999	983.855	2500049999	5142.03	10000099999	42854.6	90000299999	121510	2.5E+11
Insertion sort	0.4367	193418	0.4064	524906	0.4586	530606	1.2364	693098	1.6272	1419122	1.1465	1991890
Selection sort	27.1495	100020001	241.527	900060001	716.285	2500100001	2972.57	10000200001	31373.4	90000600001	119013	2.50001E+11
Shell sort	0.8562	145391	2.3671	454152	2.8807	808842	4.8178	1567447	14.84	5175087	22.4669	8569070
Shaker sort	71.7247	100005001	615.544	900015001	1590.98	2500025001	6641.13	10000050001	79692.6	90000150001	113827	2.5E+11
Radix sort	0.3723	9999	2.0605	29999	3.345	49999	5.6168	99999	26.8604	299999	30.7475	499999
Flash sort	0.2091	118969	0.5213	356972	1.6749	594970	2.8201	1189967	8.169	3569968	11.0708	5949968

Table 3.2. Nearly sorted data

3.2.1. Running time

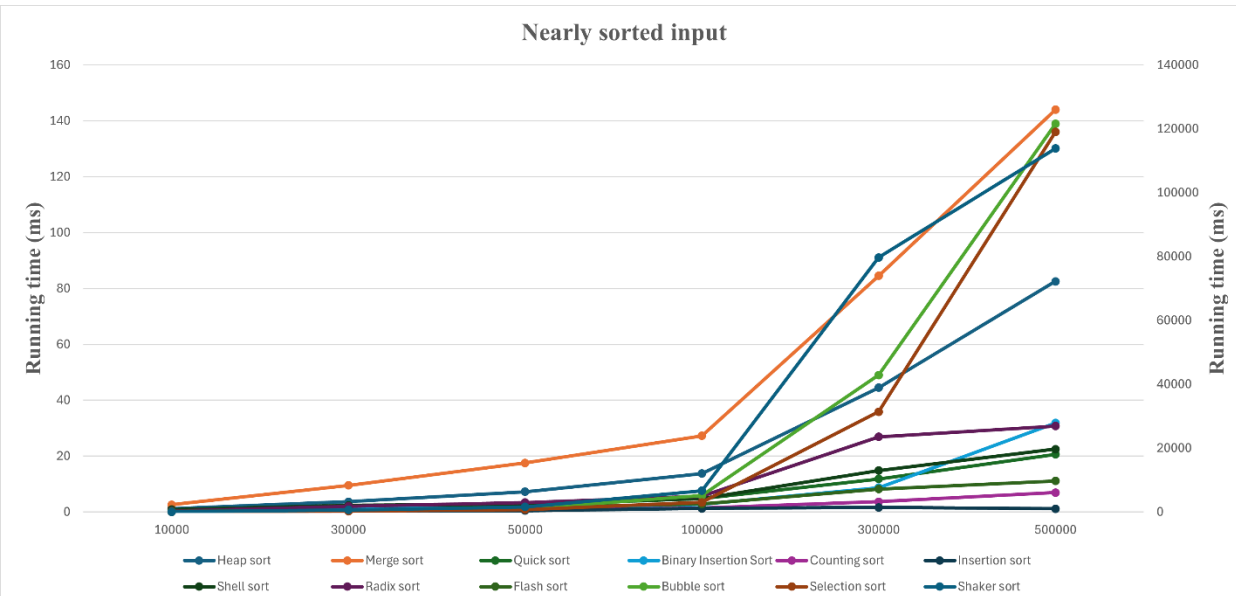

Chart 3.2.1. Nearly sorted data-running time

- **Insertion sort** is the **fastest algorithms**.
- **Selection sort**, **Bubble sort**, and **Shaker sort** remain among the **slowest algorithms**
- **Insertion sort** performs extremely well because most elements are already close to their correct positions, reducing the number of shifts and making the complexity close to **O(n)**. Quadratic algorithms still remain slow because they do not take advantage of the existing order.
- **Merge, Heap, and Quick sort** maintain stable performance since their complexity is not highly dependent on input order.

3.2.2. Number of comparisons

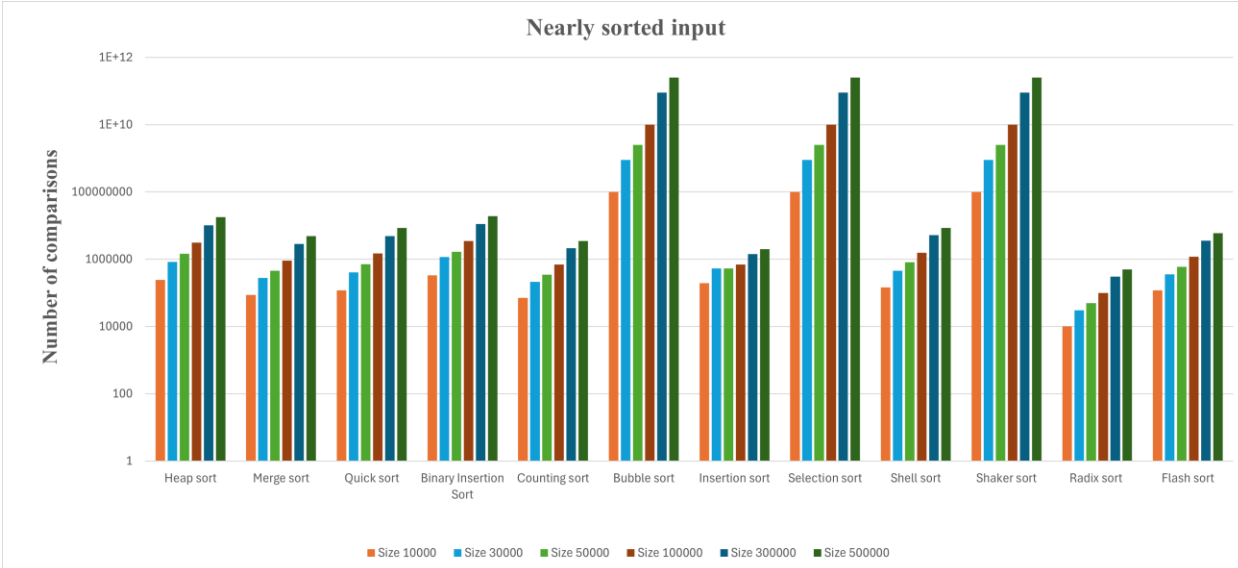

Chart 3.2.2. Nearly sorted data-number of comparisons

- **Selection sort**, **Bubble sort**, and **Shaker sort** still produce the **largest number of comparisons**.
- **Insertion sort**, **Counting sort** and **Radix sort** produce the **fewest comparisons**.
- **Insertion sort** requires far fewer comparisons because it only checks nearby elements. Quadratic algorithms still perform many comparisons regardless of input structure. **Counting and Radix sort** remain efficient with minimal comparisons.

3.3. Sorted data

Data order: Sorted Data												
Data size	10000	Size 10000	30000	Size 30000	50000	Size 50000	100000	Size 100000	300000	Size 300000	500000	Size 500000
Resulting statics	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**
Heap sort	1.035	244460	3.3059	826347	7.564	1455438	14.0329	3112517	48.7592	10279749	78.567	17837785
Merge sort	2.7727	69008	7.9244	227728	14.2061	401952	27.4193	853904	83.5124	2797264	140.622	4783216
Quick sort	0.4542	113631	0.8546	387248	2.0813	684481	4.1406	1468946	11.713	4875732	21.1951	8475732
Binary Insertion Sort	0.2314	267232	0.6305	894464	1.4684	1568928	3.5686	3337856	8.7972	10951424	14.5718	18951424
Counting sort	0.1549	70001	0.4394	210001	0.7128	350001	1.8743	700001	3.8243	2100001	5.8558	3500001
Bubble sort	38.6578	100009999	343.851	900029999	977.79	2500049999	4894.45	10000099999	45730.8	90000299999	117124	2.5E+11
Insertion sort	0.0083	29998	0.0434	89998	0.2042	149998	0.3836	299998	0.3039	899998	1.7091	1499998
Selection sort	29.0814	100020001	244.539	900060001	695.766	2500100001	2988.11	10000200001	25168.2	90000600001	87691.8	2.50001E+11
Shell sort	0.3841	120005	1.0429	390007	2.0847	700006	4.88	1500006	15.0327	5100008	21.5754	8500007
Shaker sort	72.6898	5001	595.411	900015001	1685	2500025001	6449.63	10000050001	81908.1	90000150001	113827	2.5E+11
Radix sort	0.6556	9999	1.7971	29999	4.7956	49999	4.4619	99999	25.4824	299999	30.4645	499999
Flash sort	0.4446	118993	1.1723	356993	1.2663	594993	2.7944	1189993	5.6518	3569993	9.7568	5949993

Table 3.3. Sorted data

3.3.1. Running time

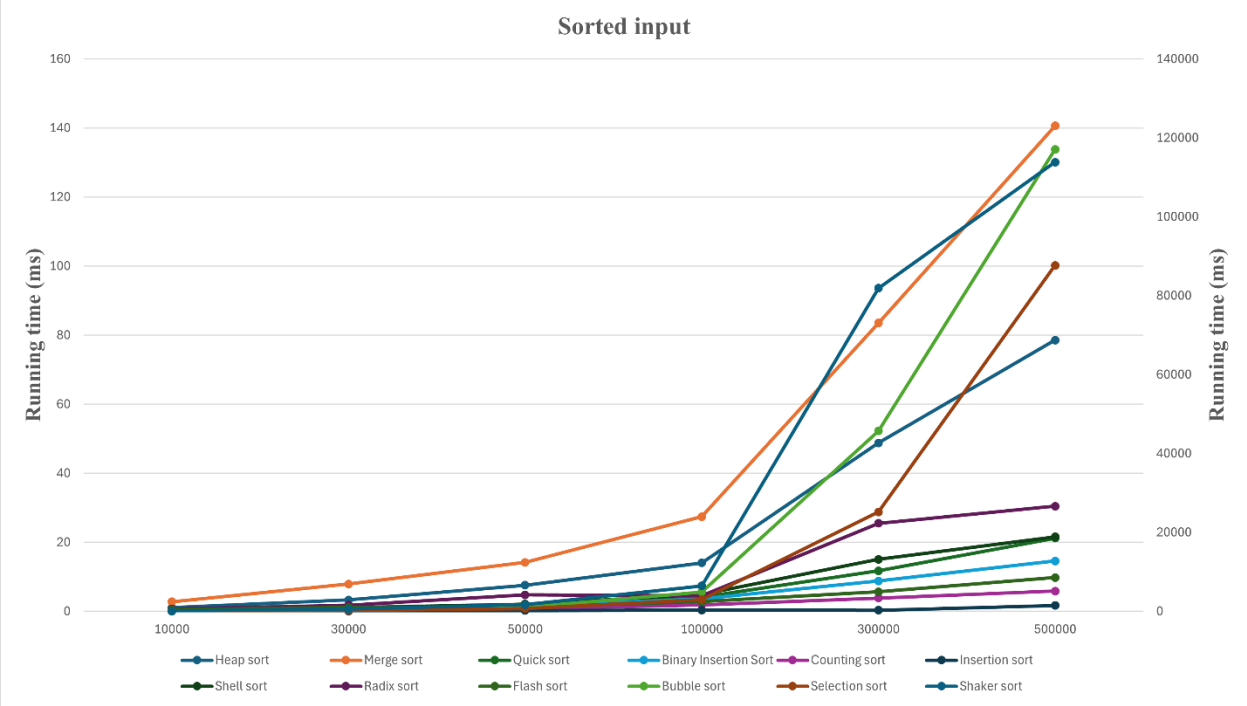

Chart 3.3.1. Sorted data-running time

- **Insertion sort** becomes one of the **fastest algorithms** when the data is already sorted.
- **Counting sort**, **Radix sort**, and **Flash sort** also maintain very **fast running times**.
- **Selection sort**, **Bubble sort**, and **Shaker sort** remain among the **slowest algorithms**.
- **Insertion sort** is highly efficient since no shifting is required, resulting in nearly **O(n)** time.
- **Selection, Bubble, and Shaker sort** remain slow because they still perform unnecessary scans and comparisons.
- **Merge, Heap, and Quick sort** show consistent performance close to **O(n log n)**.

3.3.2. Number of comparisons

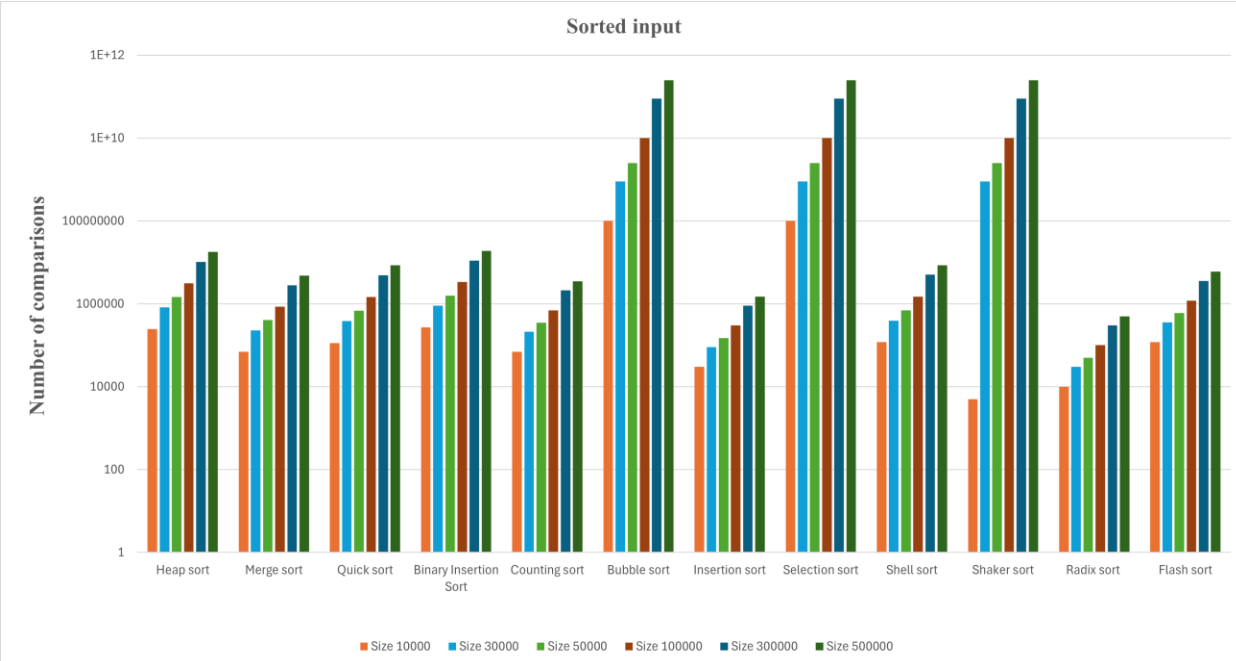

Chart 3.3.2. Sorted input-number of comparisons

- **Selection sort**, **Bubble sort**, and **Shaker sort** still produce the **largest number of comparisons**.
- **Insertion sort** performs **much fewer comparisons** compared to random or reverse sorted data.
- **Counting sort** and **Radix sort** have the **fewest comparisons**.
- **Insertion sort** performs minimal comparisons (almost one per element) since no shifting is required.
- **Selection sort** still performs the same number of comparisons regardless of input.
- **Bubble and Shaker sort** continue making repeated comparisons even when the array is already sorted leading to higher number of comparisons.

3.4. Reverse sorted data

Data order: Reverse Sorted Data												
Data size	10000	Size 10000	30000	Size 30000	50000	Size 50000	100000	Size 100000	300000	Size 300000	500000	Size 500000
Resulting statics	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**	**Running time**	**Comparison**
Heap sort	1.1596	226682	3.4916	775687	6.5041	1366047	13.8062	2926640	47.3851	9740640	74.8536	16977997
Merge sort	2.6486	64608	8.2695	219504	14.7522	382512	28.498	815024	84.6554	2678448	138.459	4692496
Quick sort	0.4654	117534	1.1274	399696	2.0608	706118	5.2083	1513481	13.2166	4994504	24.296	8699870
Binary Insertion Sort	34.8167	50242260	320.503	450819494	905.535	1251443960	3735.46	5003087890	36394.9	45010201462	110343	1.25018E+11
Counting sort	0.1357	70001	0.8881	210001	0.7209	350001	1.7594	700001	4.0683	2100001	6.3983	3500001
Bubble sort	106.753	100009999	943.748	900029999	2683.12	2500049999	11817.3	10000099999	117845	90000299999	325757	2.5E+11
Insertion sort	80.1497	100009999	826.732	900029999	2100.7	2500049999	8535.96	10000099999	197318	90000299999	212951	2.5E+11
Selection sort	37.2281	100020001	315.397	900060001	880.494	2500100001	5087.84	10000200001	32638	90000600001	112765	2.50001E+11
Shell sort	0.8217	172578	1.655	567016	3.7502	1047305	8.2533	2244585	21.958	7300919	35.7581	12428778
Shaker sort	185.314	100005001	1576.26	900015001	4083.45	2500025001	16725.4	10000050001	162616	90000150001	274028	2.5E+11
Radix sort	0.3467	9999	1.8128	29999	4.6597	49999	4.7498	99999	20.8328	299999	35.0808	499999
Flash sort	0.3186	103751	0.7823	311251	0.8788	518751	1.7303	1037501	5.0793	3112501	11.9969	5187501

Table 3.4. Reverse sorted data

3.4.1. Running time

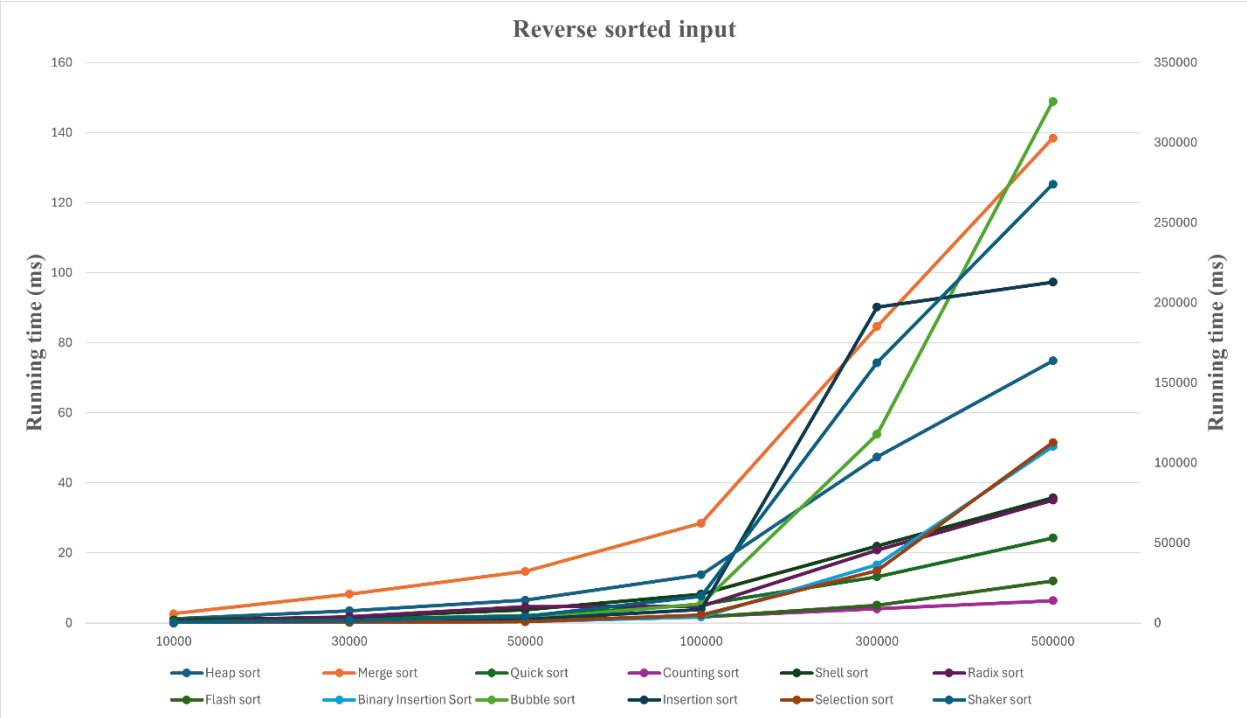

Chart 3.4.1 Reverse sorted data-running time

- **Counting sort**, **Radix sort**, and **Flash sort** remain the **fastest algorithms**.
- **Selection sort**, **Bubble sort**, and **Shaker sort** remain the **slowest algorithms**.
- This represents the **worst-case scenario** for many algorithms:
    - **Insertion sort** becomes very slow because each element must be moved across almost the entire array, leading to **O(n²)** time.
    - **Bubble, Selection, and Shaker sort** are also very slow due to repeated passes and comparisons.
    - **Merge, Heap, and Quick sort** remain efficient with **O(n log n)** complexity.
    - **Counting, Radix, and Flash sort** are still the fastest since they are not affected much by input order.

3.4.2. Number of comparisons

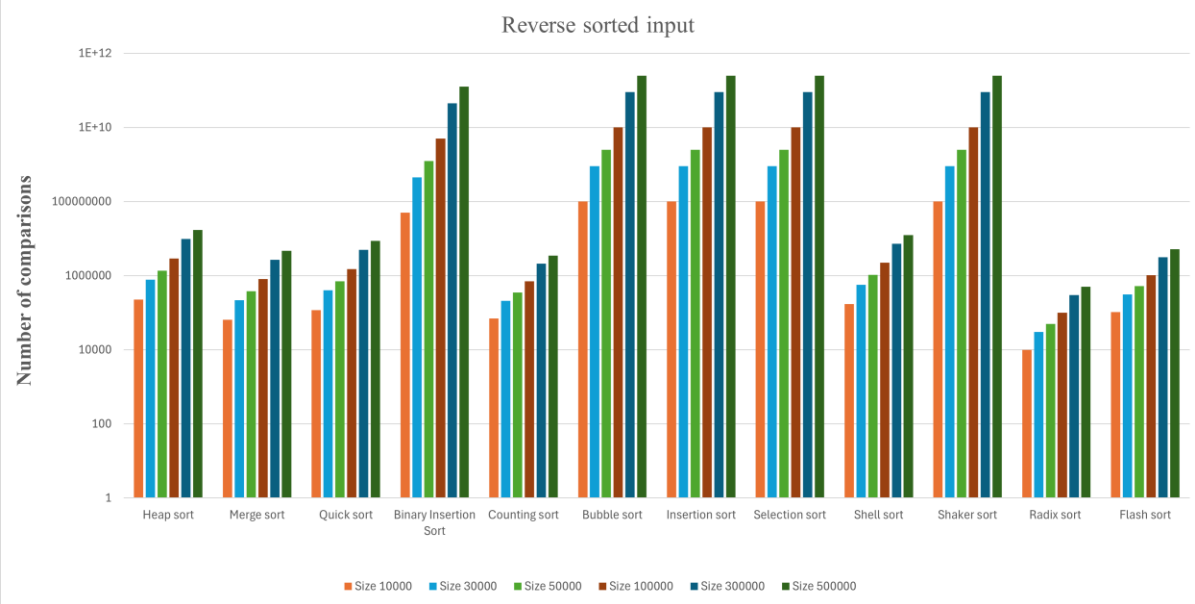

Chart 3.4.2. Reverse sorted data-number of comparisons

- **Insertion sort**, **Bubble sort**, **Selection sort**, and **Shaker sort** produce the **largest number of comparisons**.
- **Counting sort** and **Radix sort** produce the **fewest comparisons**.
- Quadratic algorithms perform the maximum number of comparisons in this case.
- **Insertion sort** also experiences a large increase in comparisons compared to other data orders.
- **Merge, Heap, and Quick sort** maintain moderate comparison growth.
- **Counting and Radix sort** still require very few comparisons.

3.5. Overall comments

- Across all data orders and data sizes, **Counting sort, Radix sort, and Flash sort are generally the fastest algorithms** because their running time grows almost linearly with the input size.
- In contrast, **Selection sort, Bubble sort, and Shaker sort are the slowest algorithms** since they have **$O(n^2)$** time complexity, causing their running time and number of comparisons to increase rapidly as the data size grows.
- Algorithms such as **Merge sort, Heap sort, and Quick sort** show **stable and efficient performance** for large datasets because their complexity is **$O(n \log n)$** and they are less affected by the initial order of the data.

- Additionally, **Insertion sort performs very well for sorted or nearly sorted data**, but becomes much slower for reverse sorted data because it reaches its worst-case complexity **$O(n^2)$**.

4. Project Organization

For the collaborative development of this Sorting Algorithms Project, GitHub was employed as the primary version control system to streamline workflow efficiency and ensure code integrity. The project's source code structure consists of the following components:

- **Header Files (.h) (4):** On2.h, Onlogn.h, On.h and DataGenerator.h
- **Source Files (.cpp) (5):** main.cpp, On2.cpp, Onlogn.cpp, On.cpp and DataGenerator.cpp

The core execution flow and command-line parsing are orchestrated within the **"main.cpp"** file. To enhance code maintainability and logical structuring, the implementations of the sorting algorithms are decoupled into three distinct source files. These files are categorized based on the theoretical average time complexities of the algorithms they contain (i.e, **$O(n^2)$** and **O(n.log n)**).

5. References

- Antti Laaksonen, "Competitive Programmer's Handbook", https://cses.fi/book/book.pdf, Accessed 2026.
- Erik Demaine, Jason Ku, Justin Solomon, "Lecture 3: Sorting notes", "Lecture 3 Recitation", "Lecture 5: Linear Sorting notes", "Lecture 5 Recitation" [Lectures Notes], https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/lecture-notes/
- GeeksforGeeks, "Sorting Algorithms", https://www.geeksforgeeks.org/sorting-algorithms/, Accessed March, 2026.
- GeeksforGeeks, " Introduction to Sorting Techniques", https://www.geeksforgeeks.org/dsa/introduction-to-sorting-algorithm/, Accessed March, 2026.
- Karl-Dietrich Neubert, "The FlashSort Algorithm", https://www.neubert.net/Flapaper/9802n.htm, Accessed March 12, 2026. (Specific to the Flash Sort algorithm discussed in the report).
- Kevin Lin, CSE 373 Winter 2026, https://courses.cs.washington.edu/courses/cse373/26wi/, Accessed March, 2026.
- Mark Allen Weiss, Short Note: Empirical study of the expected running time of Shellsort, The Computer Journal, Volume 34, Issue 1, 1991, Pages 88–91, https://doi.org/10.1093/comjnl/34.1.88

- Nguyen Ngoc Thao, “SORTING ALGORITHMS (Part I, II, III)” [PowerPoint Slides], https://drive.google.com/drive/folders/1HrEBbTf37N9D7IgtjBfFTOoxoJrNQBGa?usp=sharing
- Programiz, "Sorting Algorithms", https://www.programiz.com/dsa/sorting-algorithm, Accessed March 12, 2026.
- Robert Sedgewick, "A New Upper Bound for Shellsort", Journal of Algorithms, 1986.
- Robert Sedgewick, Kevin Wayne, "Sorting" [Lecture Notes], https://algs4.cs.princeton.edu/20sorting/, Accessed March, 2026.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, "Introduction to Algorithms", Chapters 2, 6, 7, 8, MIT Press, 2009.
- Tran Hanh Nhi, Duong Anh Duc, "Introduction to Data Structures and Algorithms", University of Science, Vietnam National University Ho Chi Minh City, 2003.